

Crowdsourced Local Disaster Alert and Resource Sharing Platform

Software Requirements Specification

Megh Patel, Madhav Yuvraj Kaushal,
Praanesh Venu Anandan, Aditi Jain

Software Application Design Lab (CSF3141)
Under the guidance of Dr. Srikanth Prabhu

October 28, 2025

Contents

1	Introduction	2
1.1	Purpose and Scope	2
1.2	System Overview	2
1.3	Stakeholders and Constraints	2
2	Software Development Lifecycle Model	3
3	Development Plan	4
3.1	Project Timeline	4
3.2	Quality Assurance	4
4	Functional and Non-Functional Requirements	4
4.1	Functional Requirements	4
4.2	Non-Functional Requirements	5
5	Software Requirements Specification	5
5.1	System Architecture	5
5.2	Data Model	6
5.3	Interface Specifications	6
5.4	Performance Specifications	6
6	UML Diagrams	7
6.1	Class Diagram	7
6.2	Activity Diagram	7
6.3	Deployment Diagram	8
7	Data Flow and Context Diagrams	9
7.1	Context Diagram	9
7.2	Data Flow Diagrams	9
8	Appendices	10
8.1	Key Performance Indicators	10
8.2	Test Scenarios	11
8.3	Data Dictionary	11
8.4	Glossary	11
	References	12

1 Introduction

The Crowdsourced Local Disaster Alert and Resource Sharing Platform is a community-driven system that enables citizens to report local disasters, validate alerts through voting, coordinate resource sharing, and receive targeted notifications. The system addresses fragmentation in disaster communication by providing geo-tagged, crowdsourced alerts with reliability scoring and geofence-based notifications.

1.1 Purpose and Scope

This Software Requirements Specification documents the platform’s requirements, architecture, and design [1]. The system addresses gaps in traditional top-down disaster response by enabling bottom-up community reporting with reliability signals and role-based moderation. The MVP scope includes user authentication, alert creation with geo-coordinates, reliability voting, resource sharing, geofenced notifications, map visualization, and administrative moderation.

1.2 System Overview

The platform implements a layered architecture with Spring Boot REST API backend, React.js SPA frontend, MySQL database with spatial extensions, and Spring Security JWT authentication. The system receives geo-tagged alerts, maintains reliability scores through community voting, evaluates geofence subscriptions, and delivers real-time updates via WebSockets. The technology stack leverages Google Maps API or Leaflet for visualization and Firebase Cloud Messaging for push notifications [2].

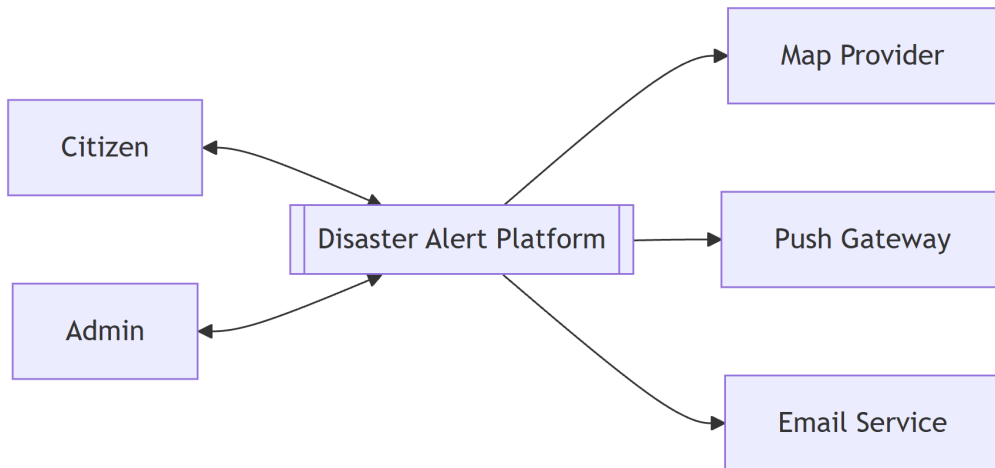


Figure 1: System context showing external actors and integration points

1.3 Stakeholders and Constraints

Primary stakeholders include citizens reporting and consuming alerts, resource providers and requesters, administrators moderating content, and local authorities integrating feeds. The system operates under constraints including privacy regulations, API rate limits for geocoding and messaging, mobile battery and bandwidth limitations, and database

spatial query throughput [3]. Key assumptions include GPS-capable devices, intermittent internet connectivity, sufficient community participation for voting convergence, and granted browser permissions for location and notifications.

2 Software Development Lifecycle Model

The platform adopts a hybrid model combining Agile iterations with incremental releases [4]. Development progresses through short 2-3 week sprints delivering vertical slices that traverse API, data, and UI layers. The model balances adaptability for geospatial and real-time features with discipline required for safety and moderation workflows.

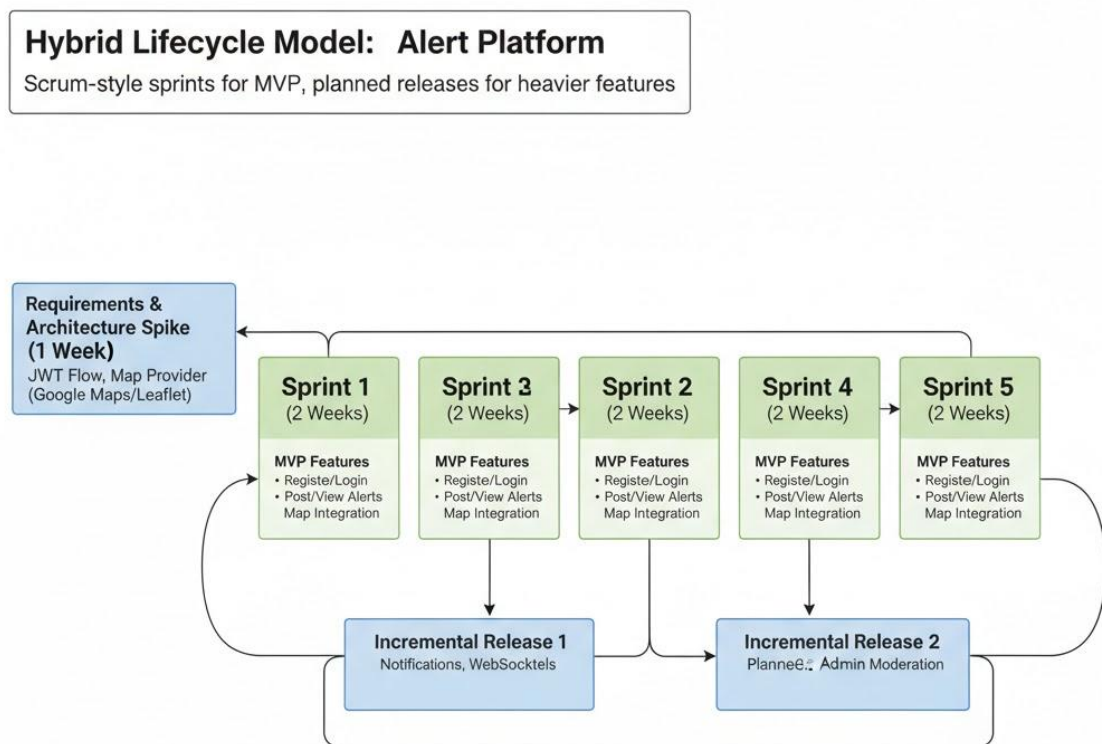


Figure 2: Hybrid Agile lifecycle model showing sprint structure and incremental releases

The model accommodates experimentation with geofencing algorithms while maintaining documentation rigor. Each sprint delivers shippable increments enabling early validation of spatial indexing and notification delivery. Quality gates between phases ensure performance and security validation before production deployment.

3 Development Plan

3.1 Project Timeline

The Phase 1 (MVP) timeline spans September-October 2025 and is structured as follows: Inception Complete takes 1 week at the very start of September; this is immediately followed by Sprint 1 - Authentication and Alert Posting taking 2 weeks (mid-September); then, Sprint 2 - Map Visualization and Voting occupies 2 weeks (late September); moving into October, Sprint 3 - Resource Sharing and Comments is allocated 2 weeks (early to mid-October); Sprint 4 - WebSockets and Geofenced Notifications is scheduled for 1.5 weeks (mid-to-late October); finally, Sprint 5 - Admin Moderation and Performance and Stabilization and Deployment are grouped together to conclude the phase in the last 1.5 weeks of October. This pacing sets a tight but generally manageable schedule by allotting a maximum of two weeks per major feature development sprint.

3.2 Quality Assurance

Quality assurance includes automated unit and integration testing achieving 70% code coverage, spatial query validation, and security testing for authentication and input validation. Acceptance criteria require P95 alert creation latency $\leq 500\text{ms}$ at 100 RPS, notification delivery within 60 seconds for 10,000 subscriptions, and map list API response within 700ms. Code reviews ensure adherence to Spring Boot and React best practices with mandatory approvals.

4 Functional and Non-Functional Requirements

4.1 Functional Requirements

FR-001: Register and authenticate users with email/phone and password using JWT sessions [5].

FR-002: Create geo-tagged alerts with type, severity, description, and optional images.

FR-003: List and view alerts with pagination, sorting, and filtering by type, severity, and time.

FR-004: Update alert metadata by authors or admins while preserving audit history.

FR-005: Display alerts on interactive map with clustering and viewport filtering [6].

FR-006: Search alerts by keywords, categories, and geo-radius filters.

FR-007: Upvote/downvote alerts once per user with real-time reliability score updates.

FR-008: Post resource offers/requests with geo-location and availability windows.

FR-009: Discover resources through filtering and map view with alert linking.

FR-010: Subscribe to notifications within user-defined geofences and topic preferences.

FR-011: Comment on alerts and resources with moderation controls.

FR-012: Review, approve, or remove flagged content with justification logged in audit records.

FR-013: Deliver push/email notifications to subscriptions whose geofences contain alert locations [7].

FR-014: Flag alerts with repeated downvotes or reports for expedited moderation.

FR-015: Enforce role-based access control (USER, VERIFIED, ADMIN) across endpoints [5].

4.2 Non-Functional Requirements

NFR-001 Performance: P95 alert creation latency $\leq 500\text{ms}$ under 100 RPS; notification evaluation completes within 60 seconds; map queries return within 700ms.

NFR-002 Scalability: Handle 10,000 active subscriptions with horizontal scaling.

NFR-003 Reliability: Maintain 99.9% monthly uptime; ensure atomic transactions for data integrity; gracefully degrade if external services fail.

NFR-004 Security: Enforce strong password policies with salted hashing; use short-lived JWT tokens with refresh flow; validate all inputs; transmit over HTTPS [5].

NFR-005 Usability: Conform to WCAG 2.1 AA for color contrast, keyboard navigation, and ARIA roles [8].

NFR-006 Maintainability: Maintain 70% test coverage; document all APIs with OpenAPI; use versioned database migrations; containerize workloads.

5 Software Requirements Specification

5.1 System Architecture

The platform implements a multi-tier architecture with clear layer separation. The Presentation Layer includes React.js SPA with responsive design. The Application Layer contains Spring Boot REST controllers and WebSocket handlers. The Business Logic Layer implements alert management, resource coordination, voting, validation, and notification services. The Data Access Layer uses repositories with spatial queries. The Data Layer leverages MySQL with spatial indexes, Redis cache, and object storage [3].

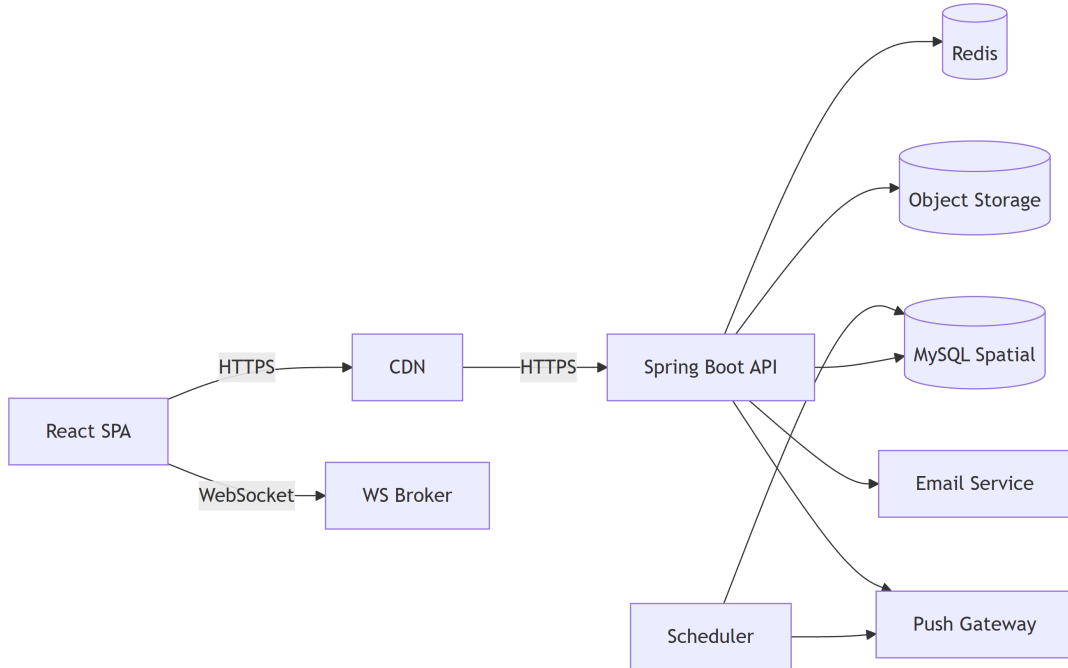


Figure 3: Multi-tier deployment architecture

5.2 Data Model

The core database schema includes entities for users, alerts, resources, votes, comments, subscriptions, and audit logs. The User table schema:

```
1 CREATE TABLE User (  
2   user_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(255) UNIQUE NOT NULL,  
5   phone VARCHAR(15),  
6   password_hash VARCHAR(255) NOT NULL,  
7   role ENUM('user','verified','admin') DEFAULT 'user',  
8   created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
9 );
```

Listing 1: User table schema with authentication fields

The Alert table includes geo-coordinates (latitude, longitude) with spatial indexing for efficient geofence queries. Supporting tables enforce one-vote-per-user constraints and maintain resource availability windows. Indexes optimize queries on location, timestamp, severity, and type fields.

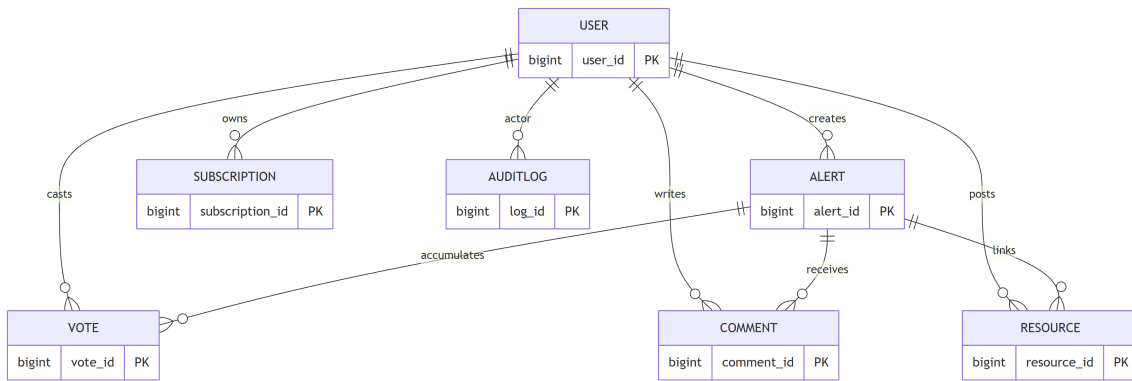


Figure 4: Entity-relationship overview of data model

5.3 Interface Specifications

REST API: POST /api/alerts creates alerts; GET /api/alerts returns paginated results with filtering; POST /api/votes registers votes; GET /api/resources queries resource availability.

WebSocket: Real-time bidirectional updates via STOMP protocol for alert creation, vote updates, and comment notifications [9].

Map API: Google Maps JavaScript API or Leaflet integration with clustered markers and spatial filtering [6].

Push Notifications: Firebase Cloud Messaging delivers geofenced notifications via HTTP/2 [7].

5.4 Performance Specifications

Processing latency breakdown: alert creation validation (100ms), database write with spatial index (200ms), geofence evaluation (2-5s), notification queueing (100ms), WebSocket broadcast (100ms), totaling 500ms end-to-end for API response. Background

notification delivery completes within 60 seconds for 10,000 subscriptions. Map queries leverage spatial indexes ensuring sub-700ms response times for 100 results.

6 UML Diagrams

6.1 Class Diagram

The Alert class serves as the primary domain entity with attributes matching the database schema (id, userId, type, severity, description, latitude, longitude, reliabilityScore) and methods for status transitions. The User class manages authentication and roles. Resource, Vote, Comment, and Subscription classes maintain relationships with integrity constraints.

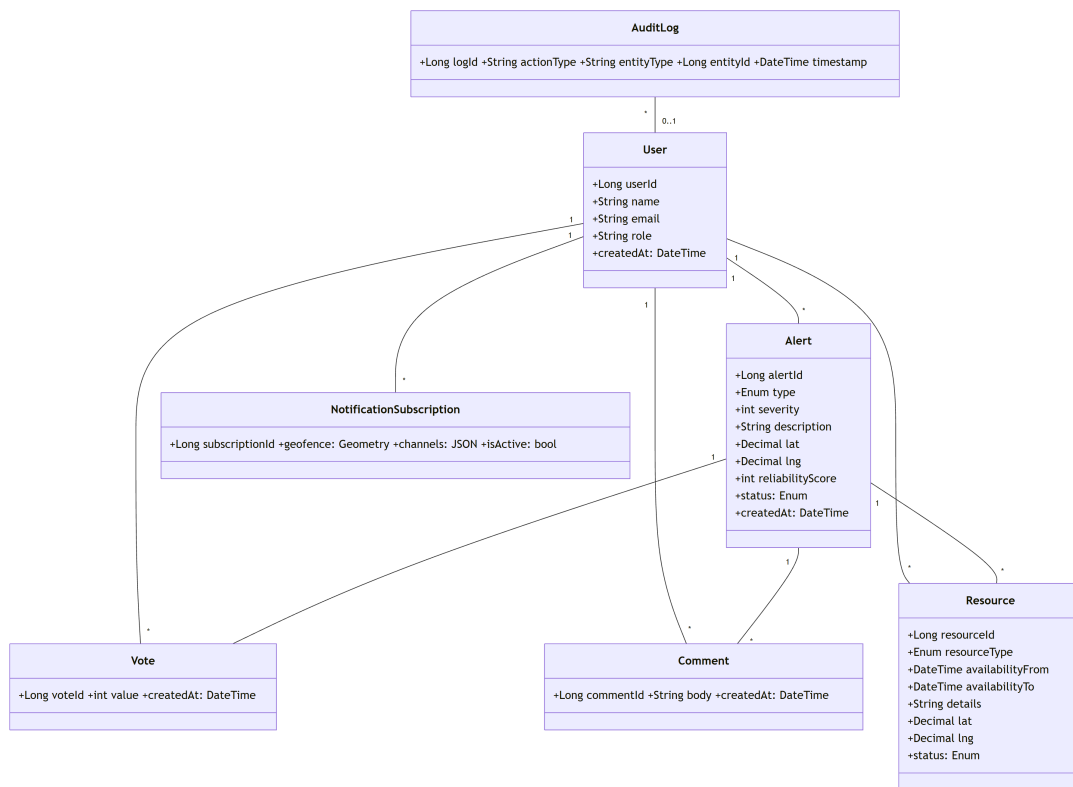


Figure 5: Core domain entities and relationships

6.2 Activity Diagram

The alert posting workflow begins with user opening the form, filling details (type, description, location, images), authentication check, API POST with JWT, backend validation, database save, search index update, real-time event emission, geofence subscription evaluation, push notification queueing, and confirmation return. Error paths handle authentication failures, validation errors, and notification delivery issues.

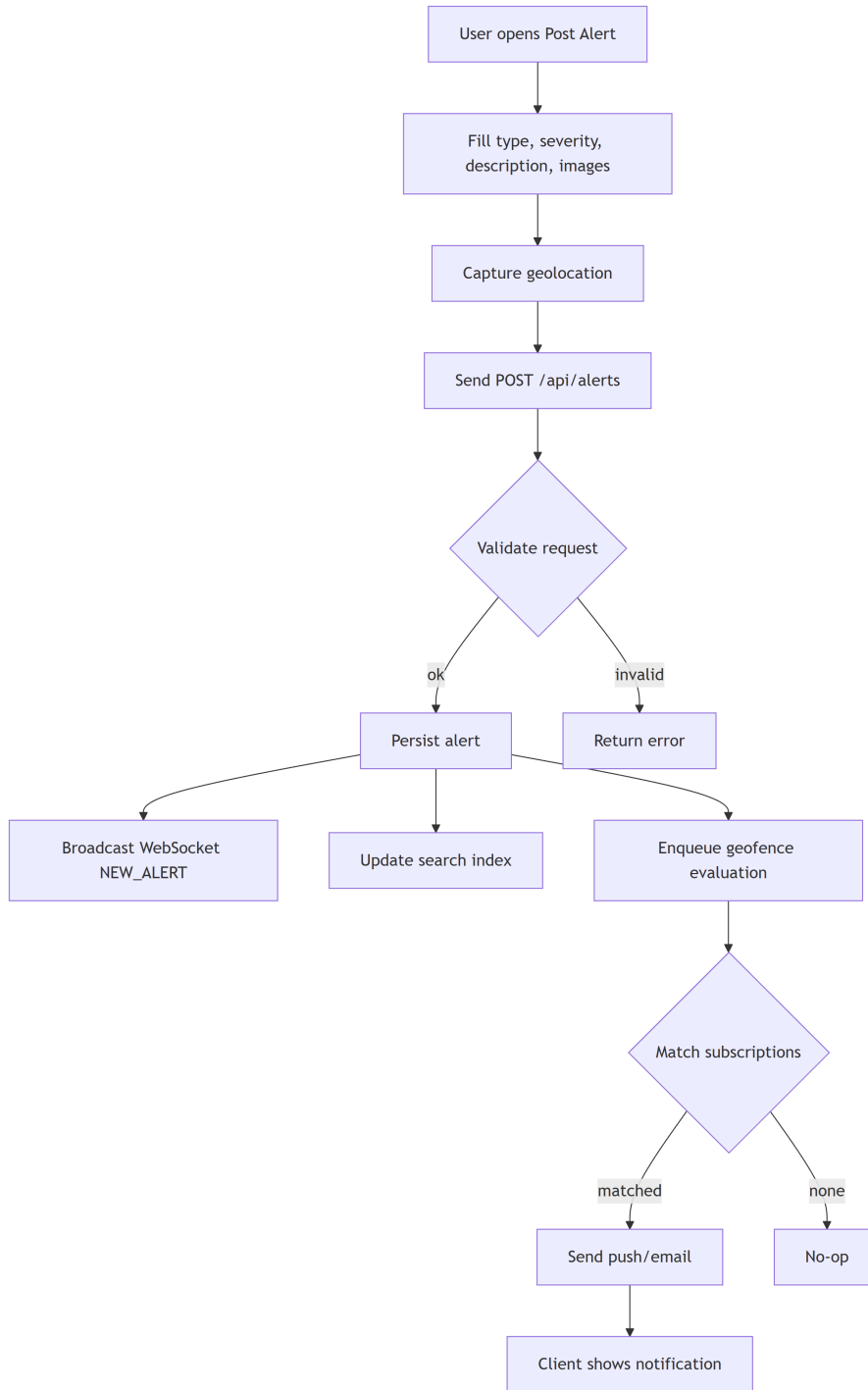


Figure 6: Activity flow from alert creation to notification delivery

6.3 Deployment Diagram

The deployment architecture spans multiple nodes: Client Browser executes React application; Edge CDN serves static assets; Application Server hosts Spring Boot instances; Database Cluster provides MySQL with replication; Cache Layer uses Redis for sessions; Object Storage holds media uploads; Push Gateway connects to Firebase Cloud Messaging. Communication protocols include HTTPS REST, WebSocket connections, and secure database connections.

7 Data Flow and Context Diagrams

7.1 Context Diagram

The context diagram (Figure 1) depicts the platform as a single process with external entities: Citizens submit and view alerts, Administrators moderate content, Map Providers supply geocoding and tiles, Push Services deliver notifications. Data flows include alert submissions with metadata, authentication tokens, map queries, moderation actions, and notification payloads.

7.2 Data Flow Diagrams

DFD Level 1 decomposes into six processes:

Process 1.0 User Management: Validates credentials, creates profiles, generates JWT tokens, logs authentication events to Audit/Logs (D5).

Process 2.0 Alert Management: Stores alerts in Alerts DB (D2) with geo-indexes, triggers validation if thresholds crossed, updates search index, publishes events.

Process 3.0 Resource Management: Stores resources in Resources DB (D3), links to alerts, enables discovery queries.

Process 4.0 Validation & Moderation: Updates reliability scores in Votes & Comments (D4), flags abuse, sends review tasks to admins.

Process 5.0 Notification Engine: Evaluates geofence subscriptions, queues push notifications via Push Service (FCM), sends delivery confirmations.

Process 6.0 Search & Reporting: Queries indexed data, returns filtered results, generates historical reports.

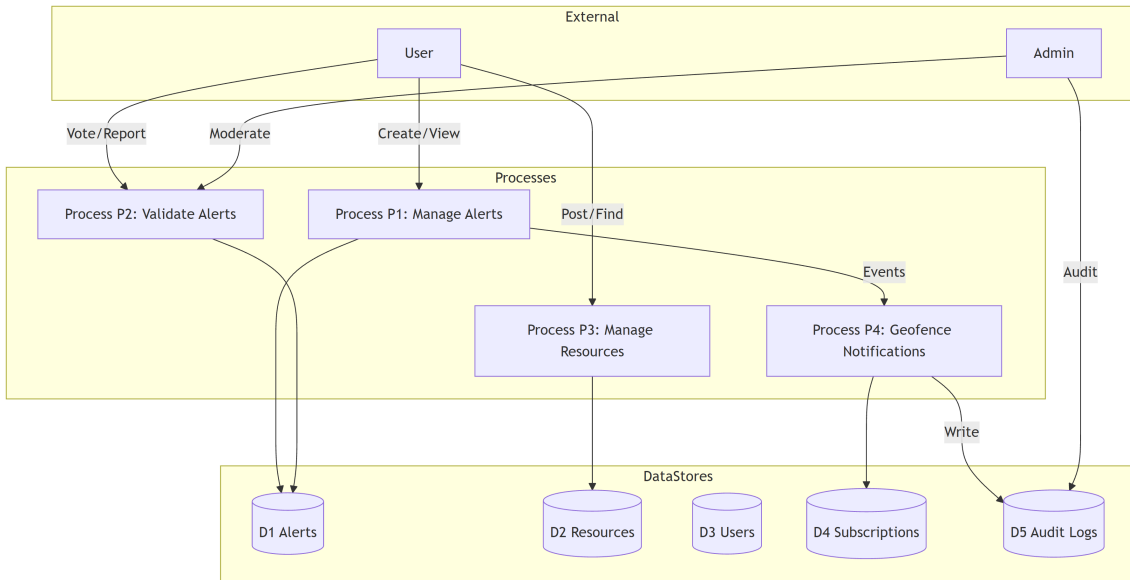


Figure 7: Level 1 DFD showing high-level processes

DFD Level 2 for Process 5.0 further decomposes into: Manage Subscriptions (5.1), Evaluate Spatial Query (5.2), Queue Notifications (5.3), and Deliver via Gateway (5.4), each with specific inputs and outputs maintaining hierarchical consistency.

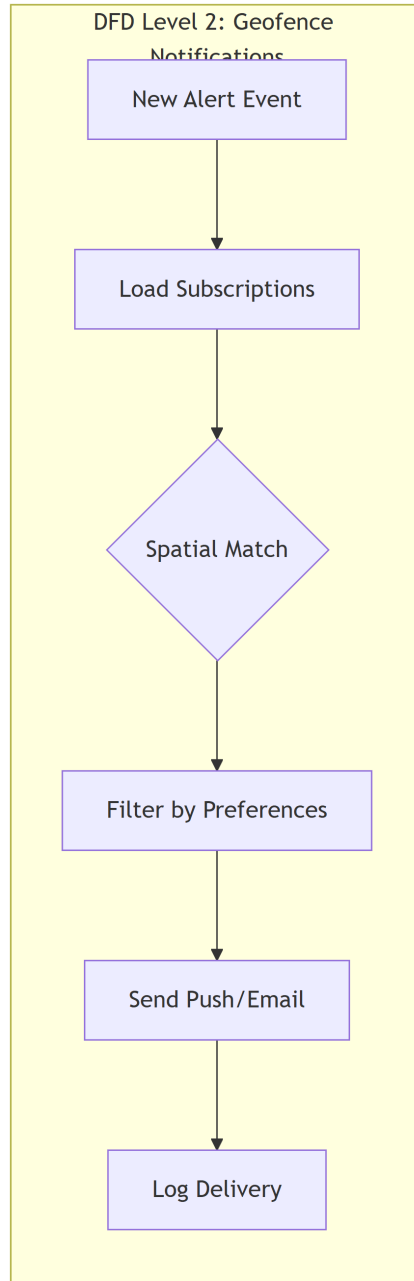


Figure 8: Level 2 DFD detailing geofence notification flow

8 Appendices

8.1 Key Performance Indicators

System effectiveness measured through: Time to First Awareness (median time from alert creation to first notification), Verification Time (median time to reach Verified status), Notification Delivery Success (ratio of successful pushes to attempts), Map Query Latency (P95 latency for list endpoint), and Moderation SLA (median time to resolve flagged content).

8.2 Test Scenarios

TS-001: User posts alert with location, verify map display within 5 seconds, confirm database storage, validate reliability score initialization.

TS-002: User upvotes alert once, verify score update in real time, confirm duplicate vote prevention, validate WebSocket broadcast.

TS-003: User subscribes to geofence, alert created inside region, verify push notification within 60 seconds, confirm delivery status.

TS-004: Admin removes flagged content with justification, verify audit log entry with timestamp and reason, confirm alert status change.

8.3 Data Dictionary

user_id: BIGINT, unique identifier for each user, primary key.

email: VARCHAR(255), user's email address, unique and not null.

alert_id: BIGINT, unique identifier for each alert, primary key.

type: ENUM('fire', 'flood', 'medical', 'power', 'other'), alert category, not null.

severity: INT (1-5), alert severity level, not null.

latitude: DECIMAL(9,6), latitude coordinate with spatial index, not null.

longitude: DECIMAL(9,6), longitude coordinate with spatial index, not null.

reliability_score: INT, calculated trust score from votes, default 0.

geofence: JSON/GEOMETRY, subscribed area for notifications, uses consistent SRID.

8.4 Glossary

API: Application Programming Interface for software integration.

JWT: JSON Web Token for stateless authentication [5].

MVP: Minimum Viable Product with core features.

SPA: Single Page Application built with React.js.

WebSocket: Full-duplex protocol for real-time updates [9].

Geofence: Spatial region defined by radius or polygon for notification targeting.

WCAG: Web Content Accessibility Guidelines [8].

FCM: Firebase Cloud Messaging for push notifications [7].

References

- [1] IEEE Computer Society, "IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications," Institute of Electrical and Electronics Engineers, 1998.
- [2] Google LLC, "Google Maps Platform Documentation," Google Cloud, 2025. [Online]. Available: <https://developers.google.com/maps/documentation>
- [3] Oracle Corporation, "MySQL 8.0 Reference Manual: Spatial Data Types," MySQL Documentation, 2025. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/spatial-types.html>
- [4] K. Beck et al., "Manifesto for Agile Software Development," Agile Alliance, 2001. [Online]. Available: <https://agilemanifesto.org>
- [5] Spring by VMware, "Spring Security Reference Documentation," Spring Framework, 2025. [Online]. Available: <https://docs.spring.io/spring-security/reference/index.html>
- [6] Leaflet, "Leaflet — an open-source JavaScript library for mobile-friendly interactive maps," 2025. [Online]. Available: <https://leafletjs.com/reference.html>
- [7] Google LLC, "Firebase Cloud Messaging Documentation," Firebase, 2025. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging>
- [8] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," World Wide Web Consortium, June 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/>
- [9] Internet Engineering Task Force (IETF), "RFC 6455: The WebSocket Protocol," December 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6455>